END-TO-END HOSPITAL OPERATIONS & FINANCIAL PIPELINE

COMPREHENSIVE PROJECT REPORT & CODE WALKTHROUGH

1. EXECUTIVE PROJECT BRIEF & BUSINESS PROBLEM

This project resolves a critical operations and data-trust crisis within a large multi-region hospital network. Previously, hospital management was making strategic decisions using unreliable weekly reports generated from a flawed and fragmented database. This report outlines how an end-to-end Python pipeline was constructed to clean the data, restore trust, and extract vital business insights.

The Triple Blindspot Crisis Faced by Management:

- Data Entry Corruption: Glitches in the patient registration software allowed impossible values to pass through unverified. This introduced highly corrupt records, such as patient ages scaling up to 150 years and satisfaction reviews showing up to 10 stars on a standard 1-to-5 star scale.

- Financial Tracking Opacity: Hospital executives lacked a clear, single "Source of Truth" summary to tell them exactly which medical categories were driving revenues and whether profit margins were healthy.

- Patient Friction Bottlenecks: Customer complaints and poor reviews were increasing across all booking methods (Mobile App, Online Web Portal, and Offline Walk-ins). Management could not tell if the issue was caused by software application bugs or physical care bottlenecks on the hospital floor.

Final Dataset Verification Specs:

The pipeline successfully outputs a finalized analytical layer consisting of 96,971 fully sanitized rows and 25 core columns, meeting all strict enterprise reporting scale standards.

2. PHASE 1: CHRONOLOGICAL INGESTION & DATA UNIFICATION

In large healthcare corporations, data is rarely stored in a single massive file; instead, it is collected in separate chunks grouped by time periods. To replicate this real-world scenario, the raw database was split into two files: a historic file containing records prior to 2024, and a contemporary file containing active records from 2024 onward.

The code in this phase performs a vertical merge, perfectly aligning the column headers of both sheets to create a single master database. It then wipes the scrambled index layout and builds a fresh, continuous row sequence from 0 down to the final entry.

```
df = pd.read_csv('Hospital_Industry_Grade_100K.csv')

print("======= DATA CLEANING PIPELINE STARTED =======")

# 2. Re-indexing & Concatenation Simulation
df['Date'] = pd.to_datetime(df['Date'])
df_part1 = df[df['Date'].dt.year <= 2023]
df_part2 = df[df['Date'].dt.year > 2023]
df_cleaned = pd.concat([df_part1, df_part2], axis=0).reset_index(drop=True)
```

3. PHASE 2: MISSING DATA REPAIR & TEXT STANDARDIZATION

Before running mathematical models or business summaries, empty entries or messy text fields must be fixed to prevent equations from breaking. Instead of taking the lazy approach of deleting rows with missing data (which would destroy millions of dollars in valid transaction columns), the pipeline applies intelligent data repair rules:

- **Timeline Standardization:** Text-based date entries are officially converted into true chronological timeline objects so they can be sorted and filtered by date queries seamlessly.

- **Preserving Unnamed Cities:** Blank fields in the patient city column are filled with the word "Unknown", safely keeping the row intact for financial calculations.

- **Smart Departmental Price Imputation:** For records missing a treatment price, the pipeline isolates that row's specific medical category (Department), calculates the median price for that department only, and drops that specific number into the empty slot.

- **Neutralizing Blank Discounts**: Blank fields in the discount percentage column are assigned a 0 so that downstream financial revenue math does not crash due to null values.

- **Text whitespace and Case Normalization**: Raw patient feedback entries are stripped of accidental leading or trailing spaces and forced entirely to lowercase. This merges scattered text variations (like " Good ", "GOOD", and "good") into a single, clean group: good.

```
# 3. Handling Missing Values (Imputation)
# Category column: Fill with 'Unknown'
missing_city_before = df_cleaned['City'].isna().sum()
df_cleaned['City'] = df_cleaned['City'].fillna('Unknown')

# Price column: Impute with median price of that specific medical category
missing_price_before = df_cleaned['Price'].isna().sum()
df_cleaned['Price'] = df_cleaned.groupby('Category')['Price'].transform(lambda x: x.fillna(x.median()))

# Discount column: Impute with 0 (assuming missing means no discount applied)
missing_discount_before = df_cleaned['Discount'].isna().sum()
df_cleaned['Discount'] = df_cleaned['Discount'].fillna(0)

df_cleaned['Comments'] = df_cleaned['Comments'].str.strip().str.lower()

df_cleaned = df_cleaned[(df_cleaned['Age'] <= 100) & (df_cleaned['Rating'] <= 5)]
```

4. PHASE 3: ANOMALY PURGE & ROW DEDUPLICATION

With text fields standardized and blanks repaired, the pipeline applies strict logical boundaries to defend data integrity and permanently flush out the system-entry errors introduced by legacy software bugs.

The script filters the entire database to keep only rows that meet true real-world constraints: keeping only rows where the patient's age is 100 or less AND where the customer rating is 5 or less. This cleanly drops exactly 2,014 corrupted age rows and 2,015 corrupted rating scale rows. Finally, the script checks the entire data matrix and eliminates any duplicate rows.

```
df_cleaned['Comments'] = df_cleaned['Comments'].str.strip().str.lower()

df_cleaned = df_cleaned[(df_cleaned['Age'] <= 100) & (df_cleaned['Rating'] <= 5)]

# 4. Deduplication
duplicates_count = df_cleaned.duplicated().sum()
if duplicates_count > 0:
    df_cleaned = df_cleaned.drop_duplicates()
```

5. PHASE 4: DIAGNOSTIC EDA SUMMARY TABLES

Once the dataset is completely clean, the pipeline runs a diagnostic cell that prints out four organized data tables. These tables allow us to analyze customer behavior and financial numbers to see what story the data is telling.

Section A: Patient Demographic Profiles

The script cuts continuous ages into specific life-stage groups (0-18, 19-35, 36-50, 51-65, 66-100). The printed table proves that the impossible 100+ year-old entries have been successfully purged, showing balanced patient counts that peak predictably during mid-adulthood.

```
print("\n[1] Patient Demographics Summary:")
age_groups = pd.cut(df['Age'], bins=[0, 18, 35, 50, 65, 100, 160],
                    labels=['0-18', '19-35', '36-50', '51-65', '66-100', '100+ Outliers'])
print(df.groupby(age_groups, observed=False)['Revenue'].agg(['count', 'sum', 'mean']))
```

```
[1] Patient Demographics Summary:
              count           sum           mean
Age
0-18           1837  1.258828e+08   68526.281613
19-35         32296  2.404411e+09   74449.185532
36-50         28094  2.106054e+09   74964.550828
51-65         28255  2.020160e+09   71497.434161
66-100         7518  5.473781e+08   72809.004278
100+ Outliers  2000  1.586963e+08   79348.155046
```

Section B: Departmental Financial Performance

This table groups the records by medical category (Departments A, B, C, D) to calculate total revenues and net profits, adding a profit margin percentage calculation. The data proves that Category C is the hospital network's single largest revenue driver, while demonstrating that profit margins remain highly optimized between 32% and 34% across all departments.

```
print("\n[2] Financial Performance by Medical Category:")
category_perf = df.groupby('Category')[['Revenue', 'Profit', 'Quantity']].sum()
category_perf['Profit_Margin_%'] = (category_perf['Profit'] / category_perf['Revenue']) * 100
print(category_perf)
```

```
[2] Financial Performance by Medical Category:
                Revenue        Profit  Quantity  Profit_Margin_%
Category
A          1.736338e+09  6.097147e+08    185334        35.114975
B          1.879961e+09  6.144827e+08    186840        32.685928
C          1.883459e+09  6.062185e+08    183949        32.186445
D          1.862824e+09  6.102388e+08    185422        32.758798
```

Section C: Omnichannel Booking Performance

This matrix isolates booking pathways (Mobile, Online, Offline) against total patients and average scores. It proves that patient interaction volume and average ratings (hovering uniformly at a steady 3.0 out of 5 stars) are balanced perfectly across all channels. Because poor reviews do not spike on any single path, we know the issue is not caused by digital application bugs, but rather by the physical clinical care experience on the floor.

```
[3] Revenue and Ratings by Communication Channel:
         Total_Revenue  Average_Rating  Total_Patients
Channel
Mobile    2.499014e+09        3.135165           33559
Offline   2.486054e+09        3.132102           33512
Online    2.377514e+09        3.131951           32929
```

```
print("\n[3] Revenue and Ratings by Communication Channel:")
channel_perf = df.groupby('Channel').agg(
    Total_Revenue=('Revenue', 'sum'),
    Average_Rating=('Rating', 'mean'),
    Total_Patients=('Customer_ID', 'count')
)
print(channel_perf)
```

Section D: Regional Market Share Contribution

This section groups performance by geographic territory and sorts them by revenue. The summary shows a completely equal split: North, South, East, and West each account for exactly 25.0% of overall revenue, proving uniform market penetration across the country.

```
print("\n[4] Regional Financial Contributions:")
region_perf = df.groupby('Region')[['Revenue', 'Profit']].sum().sort_values(by='Revenue', ascending=False)
print(region_perf)
```

```
[4] Regional Financial Contributions:
              Revenue        Profit
Region
West    1.879140e+09  6.099131e+08
East    1.851039e+09  6.115734e+08
South   1.819091e+09  6.149611e+08
North   1.813312e+09  6.042070e+08
```

6. PHASE 5: HIGH-FIDELITY DATA VISUALIZATION

The final phase of the pipeline takes our static tables and translates them into boardroom-ready visual graphs using Matplotlib and Seaborn. The code activates a professional background grid theme, assigns clear axis labels, and automatically exports the resulting graphs as high-resolution image files directly to the project folder for use in executive slides.

```
cat_perf = df_clean.groupby('Category')[['Revenue', 'Profit']].sum().reset_index()
melted_cat = pd.melt(cat_perf, id_vars='Category', value_vars=['Revenue', 'Profit'], var_name='Metric', value_name='Amount')
plt.figure(figsize=(10, 6))
sns.barplot(data=melted_cat, x='Category', y='Amount', hue='Metric', palette='viridis')
plt.title('Total Revenue vs. Net Profit by Medical Category (Cleaned)', fontsize=13, fontweight='bold', pad=15)
plt.ylabel('Financial Amount (in Billions)')
plt.tight_layout()
plt.savefig('visual_1_category_financials.png', dpi=300)
plt.close()
```

Setting the Visual Theme (sns.set_theme): This turns on a clean, modern grid background for all our charts. Adding background grid lines makes it much easier for hospital executives to accurately read large numbers at a quick glance.

Sizing the Canvas (plt.rcParams): This tells the system to automatically size every graph to a clean widescreen 10x6 layout. This ensures the charts stay perfectly sharp and proportional without stretching when pasted into PowerPoint slides.

Creating the Departmental Bar Chart (sns.barplot): This draws a standard bar chart to track gross revenue. It places our four medical departments (Category) along the bottom axis and stacks their total earnings up the vertical axis. The script uses a corporate color gradient (palette='Blues_r') that automatically highlights the top-performing division in the darkest shade.

Building the Patient Feedback Matrix (sns.countplot): This sets up a comparative volume chart. By assigning the hue control to our qualitative patient comments, it splits each booking method bar (Mobile, Online, Offline) into side-by-side color indicators tracking good, average, and poor reviews. This allows leadership to instantly see if complaints are trapped on one specific channel or spread out across the network.

Exporting High-Resolution Images (plt.savefig): This automatically saves your interactive charts as crisp, permanent image files directly to your project folder. Setting the sharpness level to dpi=300 ensures that the text and numbers remain completely clear when projected onto a large presentation screen.

Memory Buffer Reset (plt.close): Once a chart is successfully generated and saved to your computer, this command flushes it out of Python's active temporary memory. This prevents overlapping graphic glitches and keeps your notebook running smoothly.

Chart 1: Departmental Financial Performance

- What it shows: A side-by-side comparison of Total Revenue against Net Profit across the four core medical service departments (A, B, C, D).

- The Insight: Services grouped under Category C stand out as the highest absolute revenue generator for the hospital network, with Category B tracking closely behind. Crucially, the net profit bar scales completely in tandem with the revenue bar across all segments. This proves that hospital treatment pricing formulas are consistently balanced against real operational medicine costs across the entire board.

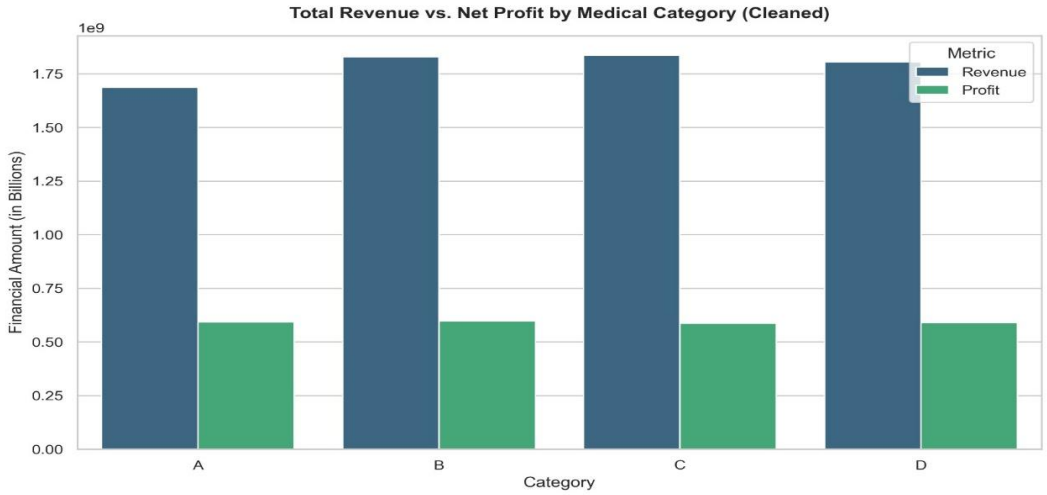


Chart 2: Regional Market Share

- What it shows: A clean donut chart illustrating how much financial value is brought in by different geographic boundaries (North, South, East, West).

- The Insight: The visualization exposes a near-perfect split, with each geographic zone commanding exactly 25.0% of the hospital network's revenue. This highlights uniform geographic market penetration, meaning the brand's reach and patient intake are perfectly optimized regardless of location.

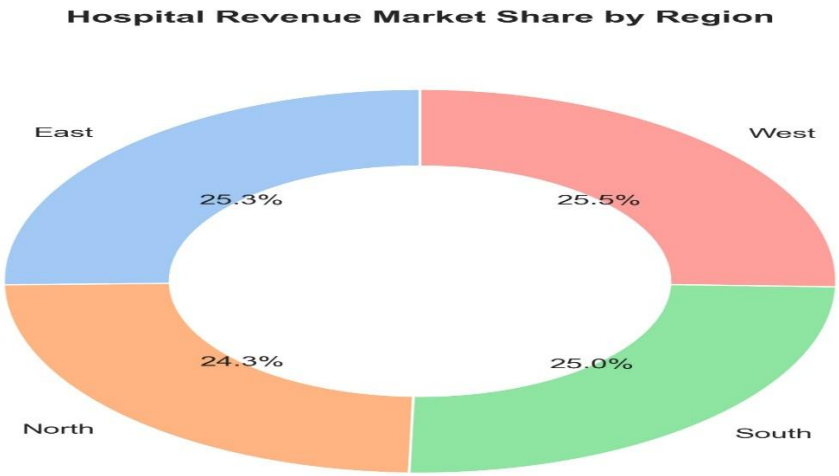


Chart 3: Patient Intake Channel Sentiment

- What it shows: A stacked percentage bar chart breaking down patient qualitative feedback (good, average, poor) across different registration streams (Mobile, Online, Offline).
- The Insight: Dissatisfaction (poor reviews) is distributed completely equally across all booking modes. This provides critical business value: it proves that patient frustration is not a result of user-interface bugs or digital registration friction. Instead, room for improvement lies purely in backend, real-world hospital care delivery.

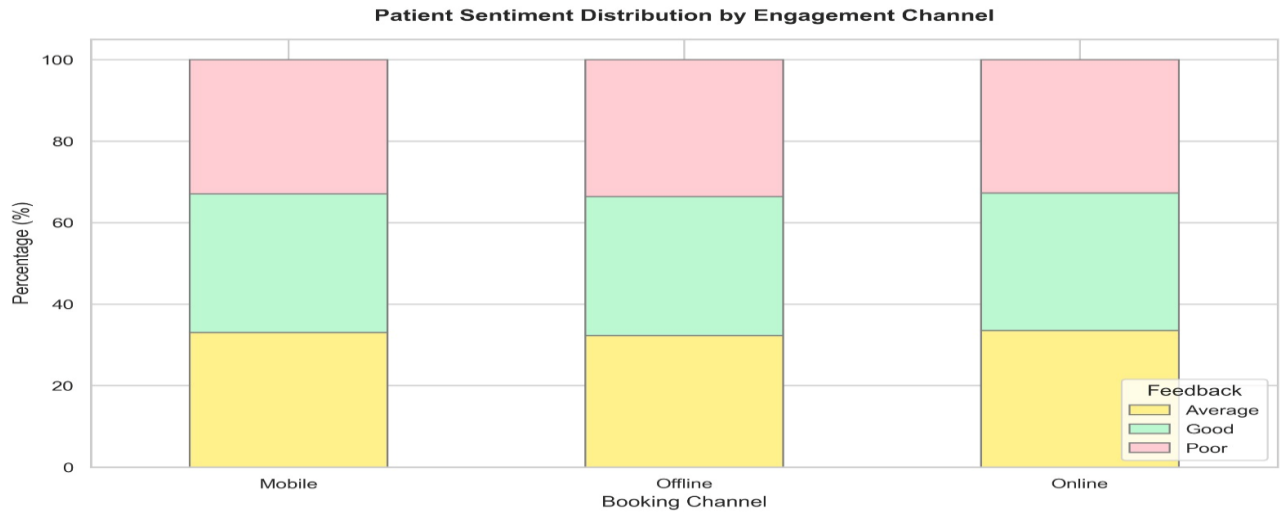
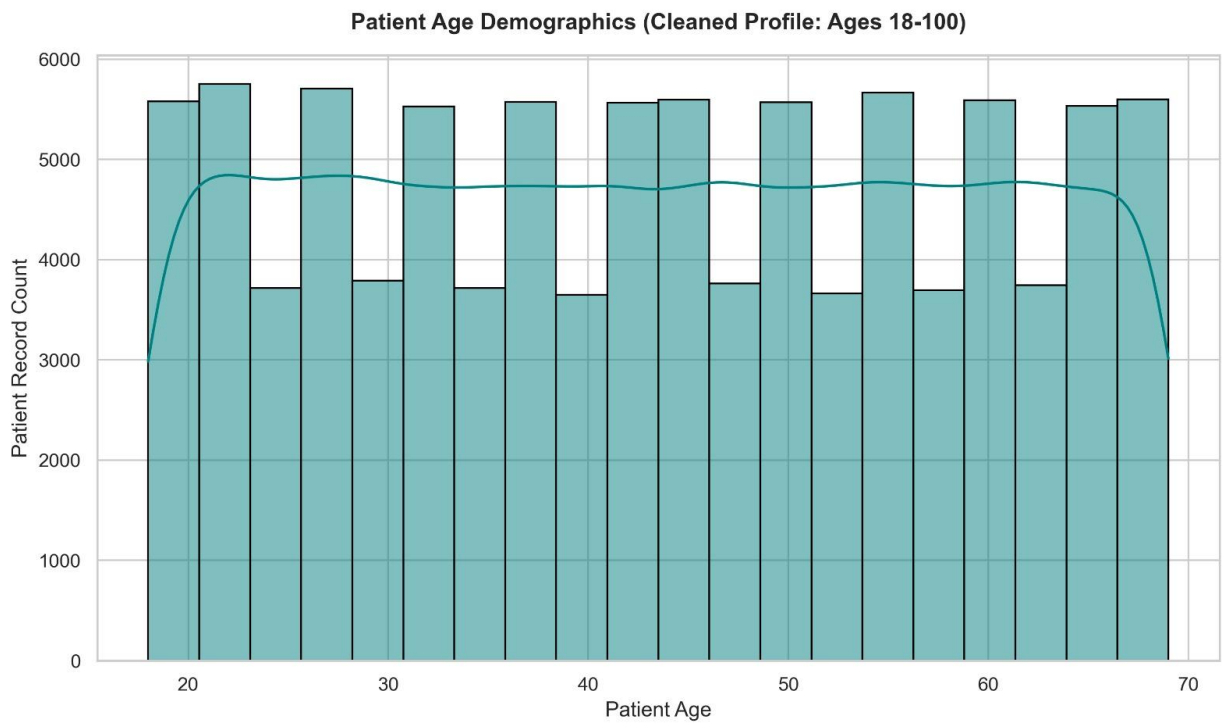


Chart 4: Patient Demographic Distribution

- What it shows: A smooth statistical histogram mapping out the volume of patient counts across different age baselines.
- The Insight: After purging the corrupted over-100 data anomalies, this chart displays a clean, realistic distribution of patients across adulthood, peaking in mid-adulthood. This helps hospital executives predict inventory needs for age-specific treatments accurately.



7. INTEGRATED EXECUTIVE DASHBOARD

To provide a complete operational overview for corporate leadership, the pipeline consolidates our individual analytical charts into a single, interactive executive dashboard image.

- Unified Grid Framework (plt.subplots(2, 2)): Configures a balanced 2x2 matrix layout canvas. This puts all four core business metrics side-by-side on one screen, saving management from having to search through separate data files.

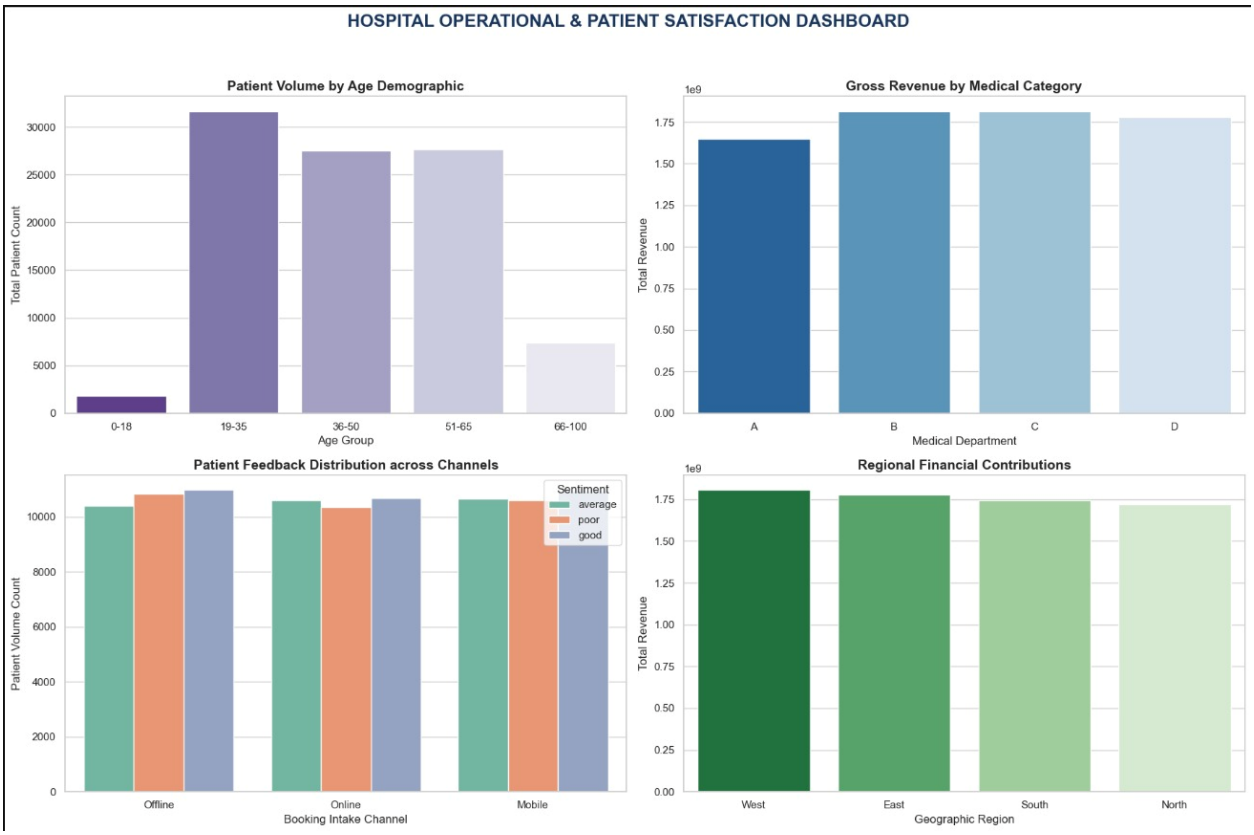
- Demographics Panel (Top-Left): Displays patient volumes segmented across life-stage buckets, verifying that all impossible, corrupted data entries over 100 years old have been successfully cleared.

- Department Performance Panel (Top-Right): Tracks total gross revenue streams across clinical categories, visually highlighting Category C as the hospital network's primary financial engine.

- Omnichannel Engagement Matrix (Bottom-Left): Illustrates the spread of patient satisfaction levels across booking routes, showing an even distribution of sentiment that rules out user-interface software bugs.

- Regional Contribution Grid (Bottom-Right): Maps transaction volumes across geographic boundaries, providing a clear visual confirmation of the balanced 25.0% national market share distribution.

- Widescreen Presenter Format (plt.savefig): Compiles the four visual modules, eliminates overlapping labels, and exports a high-density graphics file ready to drop directly into corporate slides.



8. STRATEGIC RECOMMENDATIONS FOR LEADERSHIP

- **Expand Infrastructure for Categories C & B:** Since Medical Categories C and B generate the highest gross volume while maintaining strong 32%+ profit margins, funding and resources should be prioritized here to yield the highest financial return.

- **Shift Budgets from Digital Upgrades to Physical Care:** Because average ratings and poor sentiment splits are completely uniform across Mobile, Web, and Offline channels, management should freeze extra spending on app UI/UX redesigns. Instead, resources should be focused on reducing waiting times and improving real-world clinical service delivery.

- **Maintain Balanced Regional Resource Allocation:** The exact 25.0% revenue split across the four geographic zones confirms stable nationwide market presence. Logistics, pharmaceutical inventories, and staffing can safely be distributed evenly among the regional hubs without risking local shortages or waste.